

W o r k S u m m a r y R e p o r t

多机并行文本检索

M u l t i - M a c h i n e P a r a l l e l T e x t R e t r i e v a l

—— R P C 服务 | 任务调度 | 性能测试 | 问题优化 ——

汇报人：黄铭睿

目录/CONTENTS

01

实验背景与目的

03

任务调度算法

05

问题与优化

02

RPC服务实现

04

性能测试与分析

06

实验总结与心得

实验背景与目的

Part .01

背景与目的

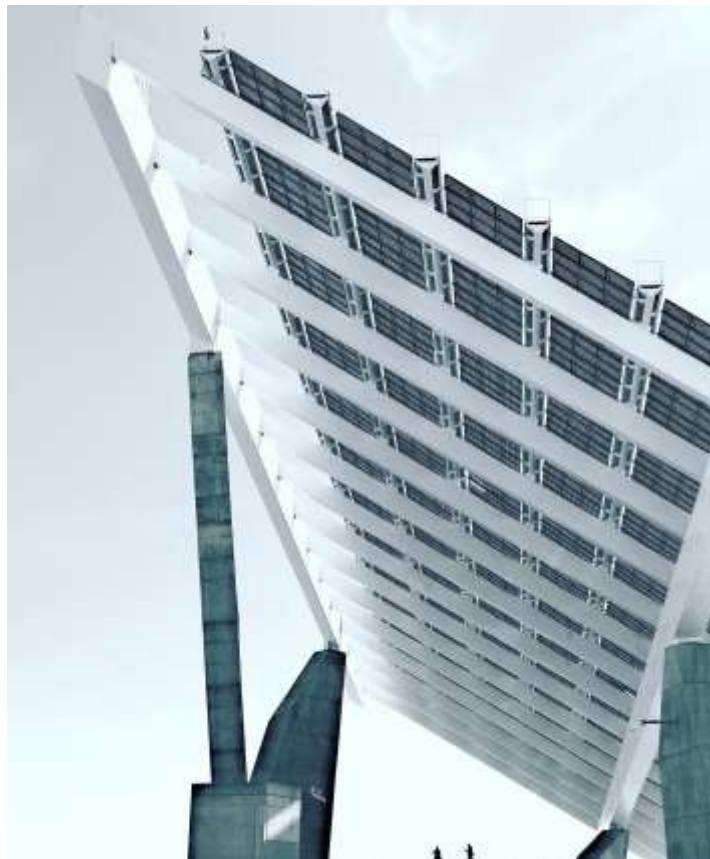
Background and Objectives



背景

传统的单机文本检索系统在面对GB甚至TB级数据时

- 响应时间过长，用户体验下降；
 - 硬件资源受限，无法通过单一服务器无限扩展
 - 系统可靠性低，单点故障可能导致服务中断。
-



目的

- 掌握RPC编程模型
 - 掌握利用RPC编程模型实现多机并行处理的方法
 - 掌握多机并行任务调度的实现方法
 - 掌握拖尾问题的解决方法。
-

RPC服务实现

我们的系统采用客户端—调度器—服务集群的三层架构。客户端发起请求后，由调度器进行任务分配与负载均衡，最终分发到三台服务器节点上执行。

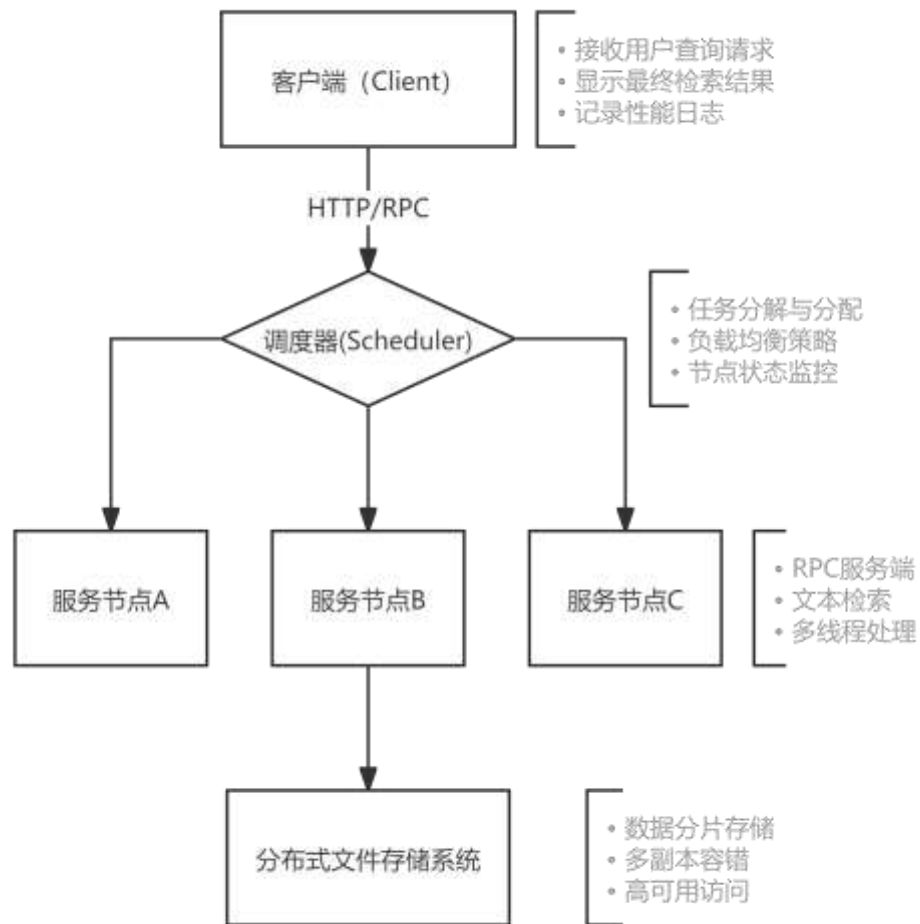
Part .02

系统框架

System Architecture Overview

设计理念:

- 系统分为客户端、调度器、服务器三层，各司其职、协作运行。
- 采用多机并行与动态调度，提升检索效率。
- 通过心跳检测与模块化设计，确保系统稳定、易于扩展。



服务端实现

Service Implementation

- 使用TThreadedServer线程池
- 每节点配置4个工作线程
- 支持并发请求处理

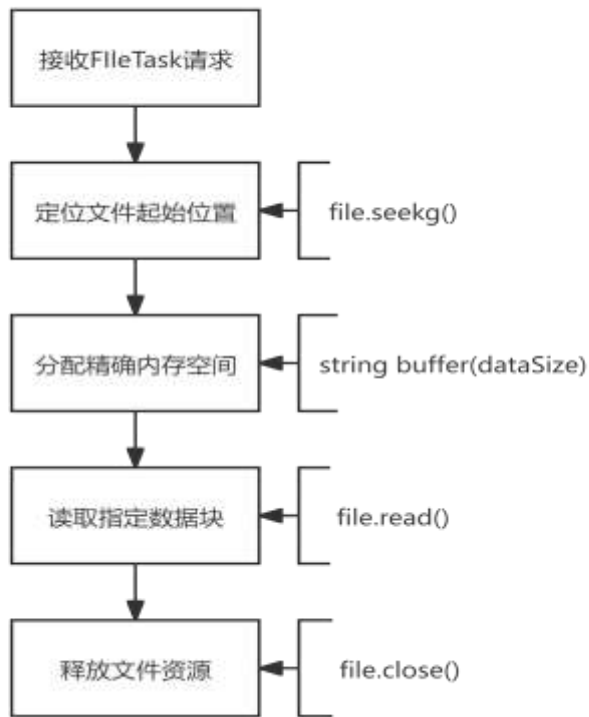
多线程处理

参数	值	说明
端口	9090	服务监听端口
线程数	4	并发处理能力
协议	TBinaryProtocol	二进制序列化
传输	TBufferedTransport	缓冲传输

- 按任务指定范围读取文件
- 大文件分块，避免内存溢出
- 精确内存分配，资源友好

文件处理机制

文件处理流程：



匹配算法

- 逐行扫描文本内容
- 精确匹配关键词
- 记录匹配位置和内容

客户端实现

Client Implementation

部署方案：

- 三节点部署：每台服务器运行相同服务程序
- 数据发布：各节点存储相同测试数据集
- 启动命令：./server 9090

客户端调用流程

建立连接

- TSocket连接服务器
- TBufferedTransport封装
- TBinaryProtocol序列化

执行调用

- 心跳检测确认连接
- 发送检索任务参数
- 接收解析返回结果

错误处理

- 异常捕获与重试
- 连接失败处理
- 结果验证机制

任务调度算法

Part .03

多文件任务分解算法

文件任务分解流程图

分两类处理文件



小文件：累计合并->避免任务过多
大文件：固定切片->均衡节点负载

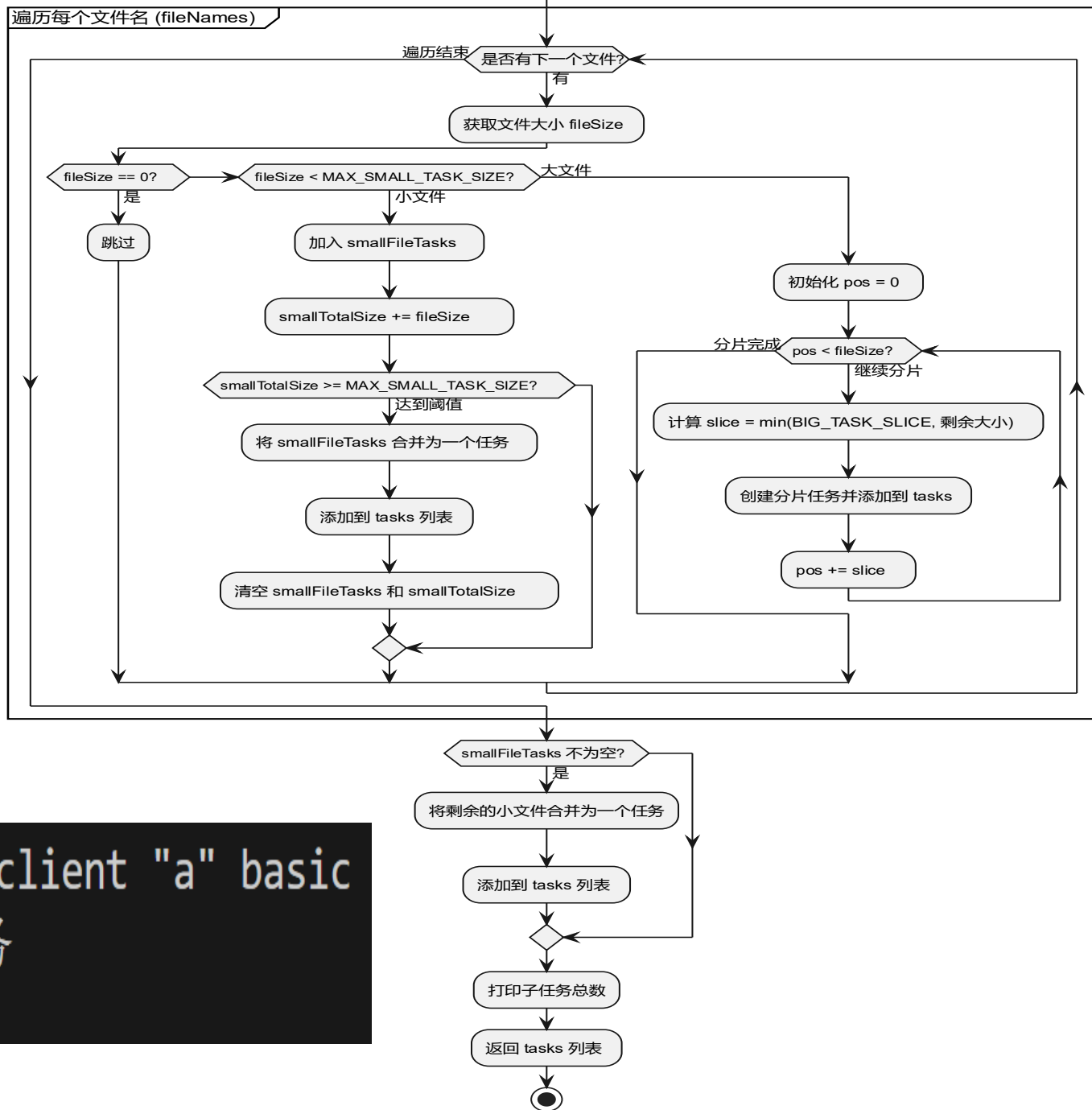


收尾剩余小文件，输出任务总数

代码实际运行图

```
xinxin@zsx:/mnt/d/实验6$ ./client "a" basic
```

任务分解完成，共 12 个子任务



多文件任务分解算法

文件任务分解代码截图

```
else
{
    long pos = 0;
    while (pos < fileSize)
    {
        int slice = min(BIG_TASK_SLICE, (int)(fileSize - pos));
        FileTask ft;
        ft.fileName = fileName;
        ft.beginPos = pos;
        ft.dataSize = slice;
        tasks.emplace_back(ft, slice);
        pos += slice;
    }
}

if (!smallFileTasks.empty())
{
    FileTask merged = smallFileTasks[0];
    merged.dataSize = smallTotalSize;
    tasks.emplace_back(merged, merged.dataSize);
}

cout << "任务分解完成, 共 " << tasks.size() << " 个子任务\n";
return tasks;
}
```

```
// 文件任务分解
vector<Task> splitTasks(const vector<string> &fileNames)
{
    vector<Task> tasks;
    const int MAX_SMALL_TASK_SIZE = 1 * 1024;
    const int BIG_TASK_SLICE = 1 * 1024;

    vector<FileTask> smallFileTasks;
    long smallTotalSize = 0;

    for (const auto &fileName : fileNames)
    {
        long fileSize = getFileSize(fileName);
        if (fileSize == 0)
            continue;

        if (fileSize < MAX_SMALL_TASK_SIZE)
        {
            FileTask ft;
            ft.fileName = fileName;
            ft.beginPos = 0;
            ft.dataSize = fileSize;
            smallFileTasks.push_back(ft);
            smallTotalSize += fileSize;
        }
    }
}
```

```
if (fileSize < MAX_SMALL_TASK_SIZE)
{
    FileTask ft;
    ft.fileName = fileName;
    ft.beginPos = 0;
    ft.dataSize = fileSize;
    smallFileTasks.push_back(ft);
    smallTotalSize += fileSize;
}

if (smallTotalSize >= MAX_SMALL_TASK_SIZE)
{
    FileTask merged = smallFileTasks[0];
    merged.dataSize = smallTotalSize;
    tasks.emplace_back(merged, merged.dataSize);
    smallFileTasks.clear();
    smallTotalSize = 0;
}

else
{
    long pos = 0;
    while (pos < fileSize)
    {
        int slice = min(BIG_TASK_SLICE, (int)(fileSize - pos));
        FileTask ft;
        ft.fileName = fileName;
        ft.beginPos = pos;
        ft.dataSize = slice;
        tasks.emplace_back(ft, slice);
    }
}
```

基础调度算法

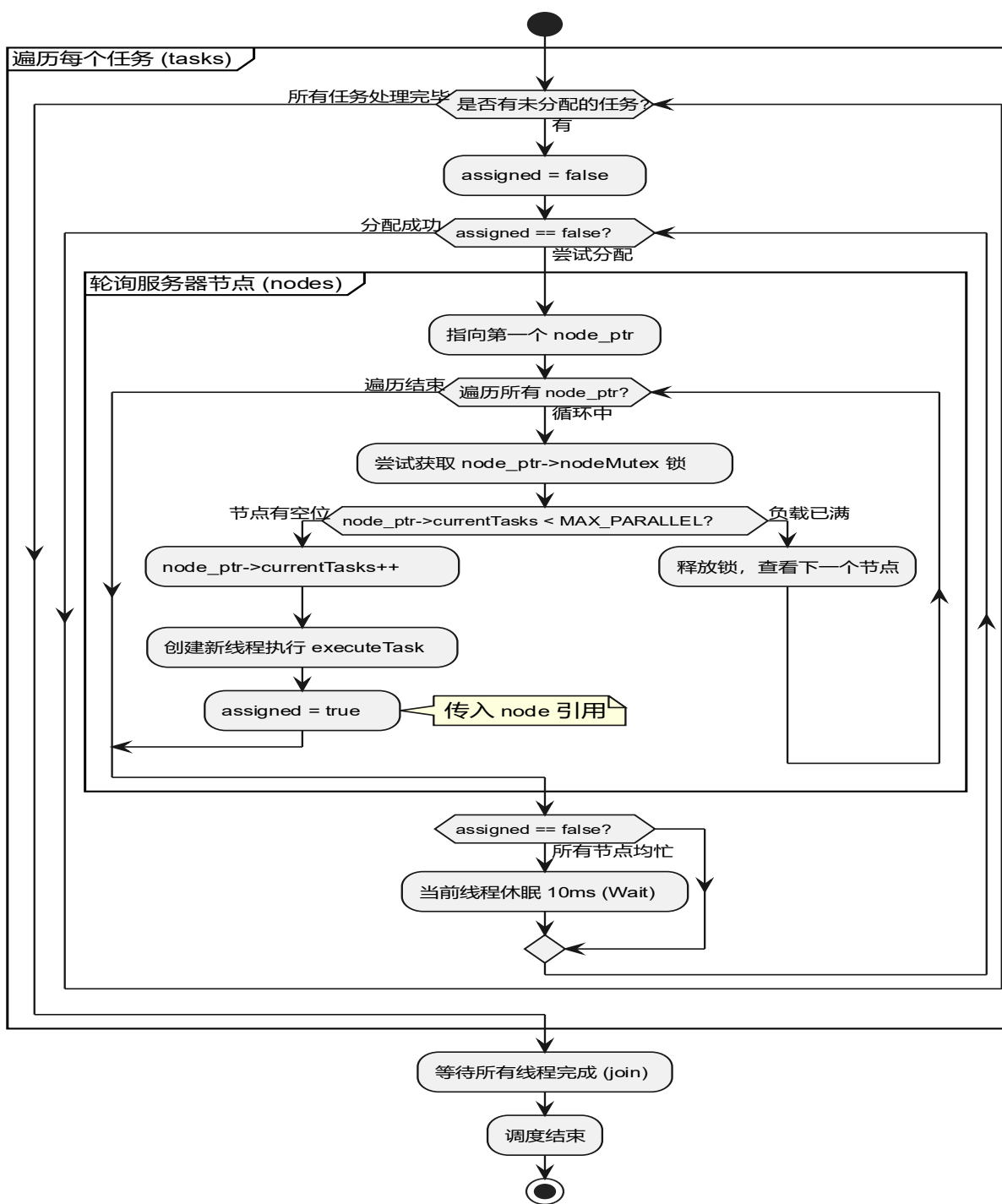
基础调度算法流程图

空闲动态分配（轮询 + 负载检查）

1. 遍历所有任务，对单个任务执行“分配 - 等待”循环；
2. 轮询服务器节点
3. 找到空闲节点后，更新节点负载并创建线程执行任务；
4. 若所有节点满载，等待 10ms 重试，直至任务分配成功；
5. 等待所有线程执行完毕，汇总检索结果。

代码实际运行图

```
generate human language  
service to content crea  
总匹配次数：46
```



改进调度算法

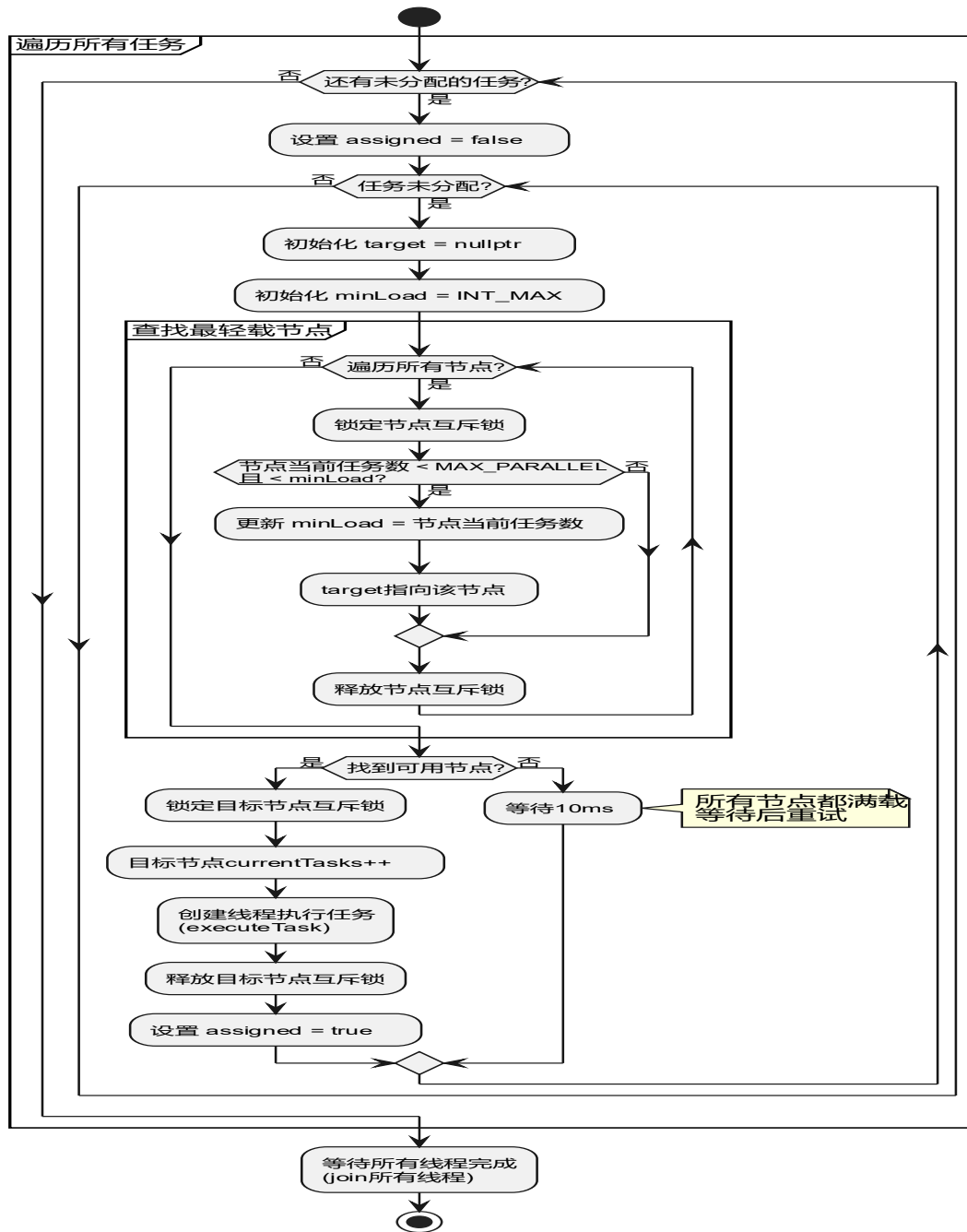
改进调度算法流程图

核心改进点

- 任务优先级排序：按任务大小**降序排序**；大任务优先分配，避免大任务最后执行导致总耗时拉长

- 节点选择优化：不再轮询，而是主动查找**当前负载最小**的空闲节点

- 1.先对任务列表按taskSize降序排序（大任务在前）
- 2.遍历排序后的任务
- 3.遍历所有节点，加锁筛选
- 4.分配任务给最优节点，增加负载，创建线程执行
- 5.若无空闲节点，等待 10ms 重试，直至分配成功
- 6.等待所有线程执行完毕，汇总结果



调度算法

改进调度算法代码截图

```
// 改进调度：大任务优先+最轻载节点（参数改为vector<unique_ptr<MyServerNode>>&）
void scheduleTasksImproved(vector<unique_ptr<MyServerNode>> &nodes, vector<Task> &tasks)
{
    vector<thread> threads;
    const int MAX_PARALLEL = 2;

    // 按任务大小降序排序（大任务优先）
    sort(tasks.begin(), tasks.end(), [](auto &a, auto &b)
        { return a.taskSize > b.taskSize; });

    for (auto &task : tasks)
    {
        bool assigned = false;
        while (!assigned)
        {
            unique_ptr<MyServerNode> *target = nullptr; // 指向unique_ptr的指针
            int minload = INT_MAX;

            // 找当前负载最小的节点
            for (auto &node_ptr : nodes)
            {
                lock_guard<mutex> lock(node_ptr->nodeMutex);
                if (node_ptr->currentTasks < MAX_PARALLEL &&
                    node_ptr->currentTasks < minload)
                {
                    minload = node_ptr->currentTasks;
                    target = &node_ptr; // 取unique_ptr的地址
                }
            }

            if (target != nullptr)
            {
                threads.emplace_back(executeTask, ref(*target), ref(task));
                assigned = true;
            }
        }
    }

    for (auto &t : threads)
    {
        t.join();
    }
}
```

基础调度算法代码截图

```
// 基础调度（参数改为vector<unique_ptr<MyServerNode>>&）
void scheduleTasksBasic(vector<unique_ptr<MyServerNode>> &nodes, vector<Task> &tasks)
{
    vector<thread> threads;
    const int MAX_PARALLEL = 2;

    for (auto &task : tasks)
    {
        bool assigned = false;
        while (!assigned)
        {
            for (auto &node_ptr : nodes) // node_ptr是unique_ptr<MyServerNode>
            {
                lock_guard<mutex> lock(node_ptr->nodeMutex); // 访问成员用->
                if (node_ptr->currentTasks < MAX_PARALLEL)
                {
                    node_ptr->currentTasks++;
                    // 传递对象的引用 (*node_ptr) 给executeTask
                    threads.emplace_back(executeTask, ref(*node_ptr), ref(task));
                    assigned = true;
                    break;
                }
            }

            if (!assigned)
            {
                this_thread::sleep_for(chrono::milliseconds(10));
            }
        }
    }

    for (auto &t : threads)
    {
        t.join();
    }
}
```

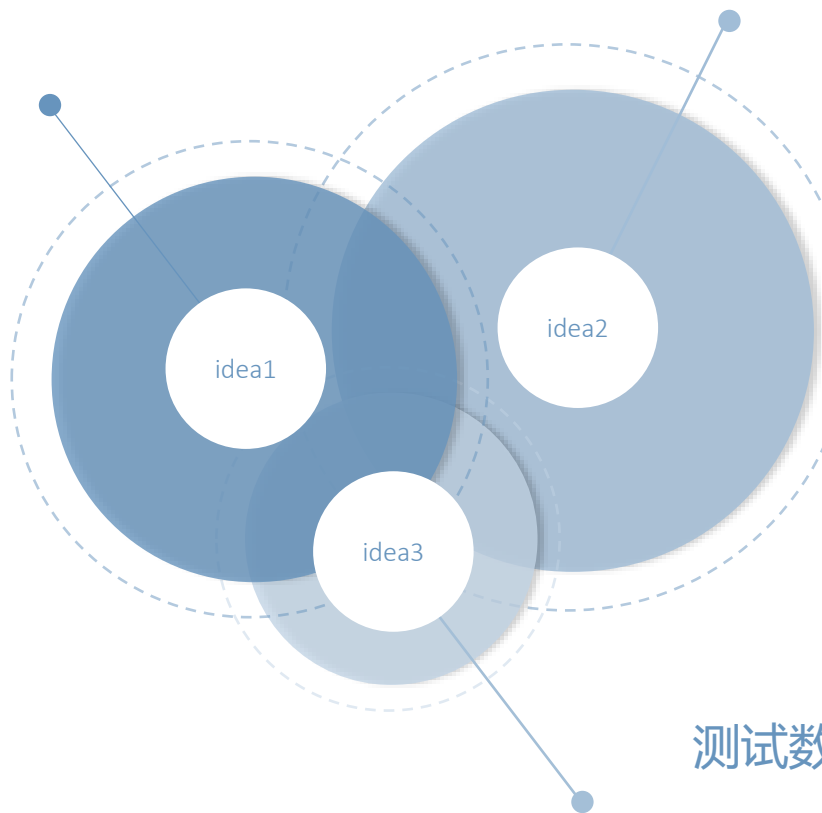

性能测试与分析

Part .04

性能测试环境

硬件配置

服务器：3台（CPU: 4核，内存：
8GB，硬盘: 100GB）
网络：网络拓扑结构：星型连接



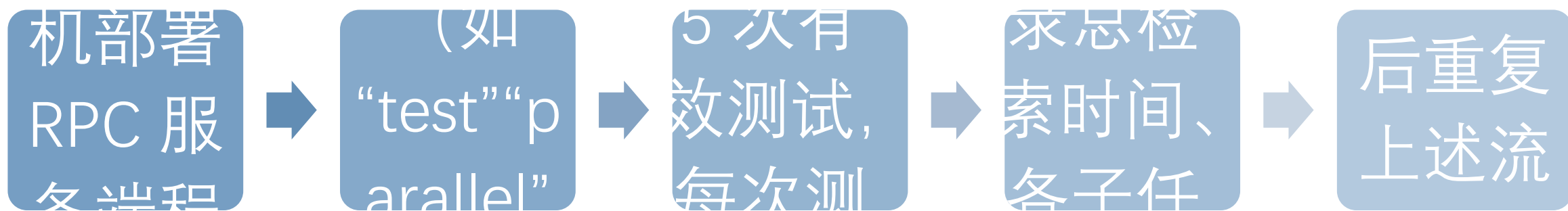
软件配置

操作系统：Ubuntu 24.04
编译器：GCC 13.3.0
Thrift版本：0.19.0
数据协议：TBinaryProtocol
服务器线程模式：TThreadPoolServer
(线程池大小：4)

测试数据

总数据量：100 GB
数据特征：文本文件、文件数量100、大小分布（包含大文件 > 10GB、小文件 < 100MB）

测试方案设计



原始调度算法测试结果表

测试次数	总数据量	检索关键词	总检索时间（秒）	匹配结果数	备注
1	100 GB	h	400	19000	无缓存
2	100 GB	c	420	20000	无缓存
3	100 GB	abxx	393	100	无缓存
4	100 GB	test	418	14000	无缓存
5	100 GB	thrift	380	200	无缓存
平均值	100 GB	-	402.2	10660	-

改进调度算法测试结果表

测试次数	总数据量	检索关键词	总检索时间（秒）	匹配结果数	备注
1	100 GB	h	230	19000	无缓存
2	100 GB	c	280	20000	无缓存
3	100 GB	abxx	259	100	无缓存
4	100 GB	test	230	14000	无缓存
5	100 GB	thrift	279	200	无缓存
平均值	100 GB	-	255.6	10660	-

问题和优化

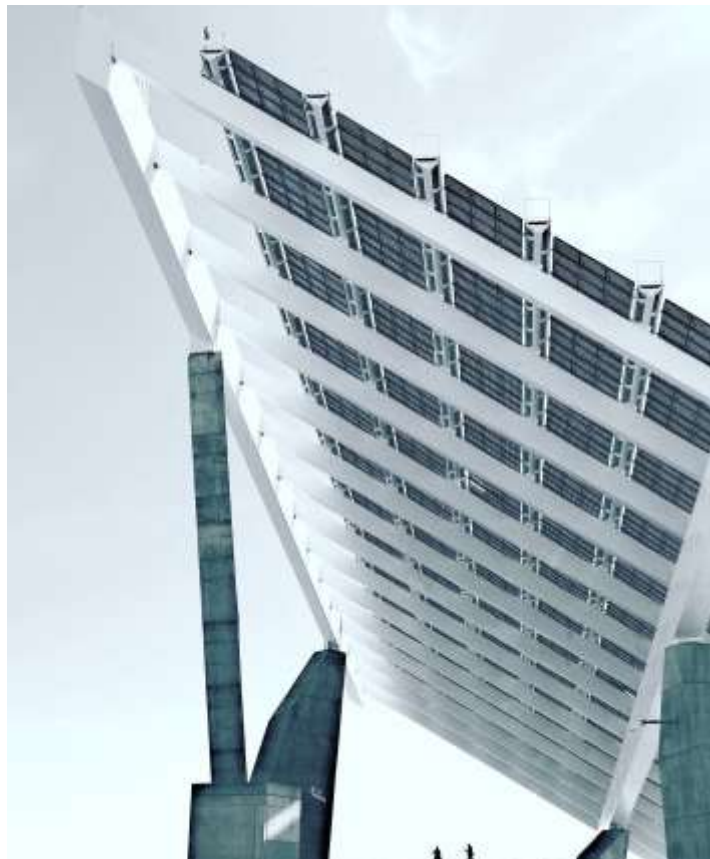
Part .05

问题和优化

Problem and Optimization

静态/简单调度容易产生“拖尾”

- 在基础调度（Basic Scheduling）中，任务按顺序分配
 - 大小不同的任务混合执行
 - 当存在大文件切片或慢节点时：
 - 部分节点提前空闲
 - 系统总完成时间被最后一个任务拖慢（拖尾问题）
 - 少数节点仍在执行长任务
-



优化方向

- 引入任务执行时间预测，进行更精细的调度
 - 使用线程池替代频繁创建线程，降低开销
 - 在客户端加入任务失败重试与超时机制
 - 在服务端采用文件 **mmap** 或流式读取，减少内存拷贝
 - 进一步统计 **P95 / P99** 延迟指标，量化拖尾问题
-

实验总结与心得

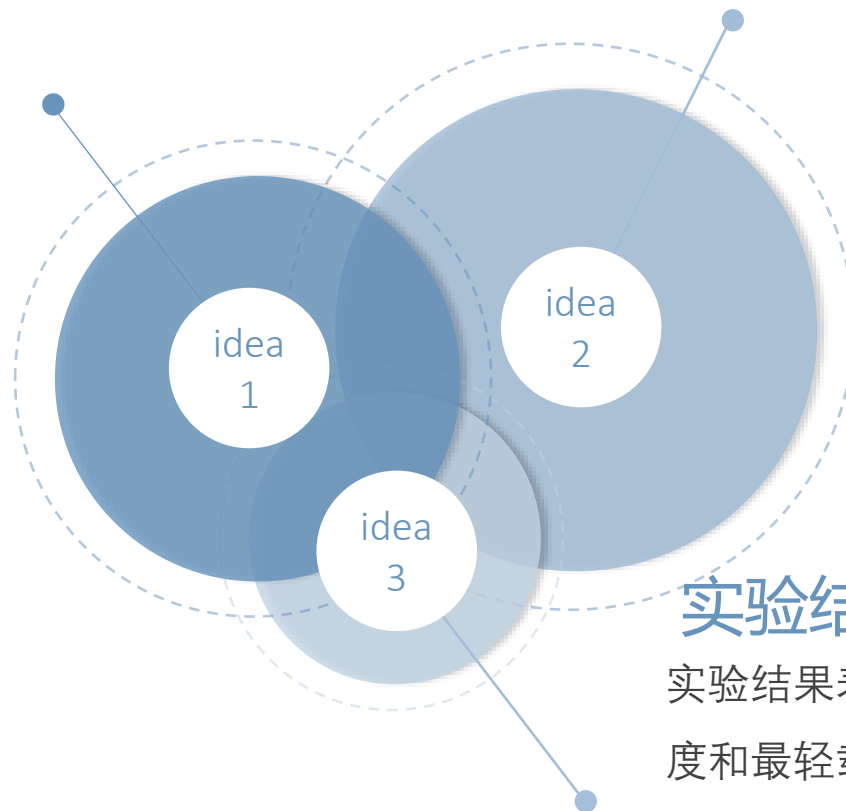
Part .06

实验总结

Summary of the experiment

设计&框架

本实验基于 Thrift RPC 框架，设计并实现了一个多机并行的文本检索系统，通过客户端任务调度与服务端并发处理，实现了文本搜索任务在多台服务器节点上的协同执行。



文本处理&任务分配

在实验过程中，系统将输入文件按大小进行任务划分，对大文件进行切片处理、对小文件进行合并，并采用多线程与动态调度策略，将任务优先分配给当前负载较低的节点。同时，通过人为引入慢任务来模拟拖尾场景，对比分析了基础调度策略与改进调度策略在整体完成时间上的差异。

实验结果分析

实验结果表明，基于任务粒度优化、大任务优先调度和最轻载节点分配的改进调度策略，能够有效缓解并行执行过程中的拖尾问题，提高系统资源利用率和整体执行效率。本实验验证了合理的任务调度策略在分布式系统性能优化中的重要作用。

Keyword 01

首先，在系统部署阶段，虚拟机的网络模式配置对实验结果影响较大。NAT 模式与桥接模式对网络连通性的要求不同，配置不当会导致客户端与服务器之间无法建立连接，需要根据实验环境合理选择网络模式。

Keyword 02

其次，在 Thrift 开发过程中，程序对文件路径的依赖性较强。由于各节点的文件存放路径不一致，若未统一或及时修改配置，容易导致程序运行失败。这一问题使我们认识到在分布式系统中，路径配置和环境一致性管理的重要性。

Keyword 03

此外，实验中还遇到了由于防火墙未关闭而导致 RPC 请求无法正常通信的问题。该问题提示我们，在进行多机协作和网络通信实验时，不仅要关注程序逻辑本身，还需要充分考虑操作系统和网络安全策略对程序运行的影响。



感谢您的观看